Graphs and Search

Topic 3 of 10

11 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Friend Graph with Timeline (Friend / Unfriend Events)

HARD | asked 2026-09-29 | graph, union-find

Examples: Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] -> answer 5.

```
def earliest_all_connected(n, events):
```

2. Template Variable Expansion with Follow-Ups

MEDIUM | asked 2026-10-17 | string, parsing, graph, topological-sort, dfs, cycle-detection

Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion. Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value. Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.

```
def expand_template(text, mapping):
```

3. Train / Flight Schedule Reachability

MEDIUM | asked 2026-10-08 | graph, bfs, shortest-path

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS. Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.

```
def earliest_arrivals(legs, source, strict=True
  ):
```

4. Course Scheduling and Dependency Variants

MEDIUM | asked 2026-10-04 | graph, topological-sort, cycle-detection

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route. Course-ordering variant Work through the course-ordering problem, including validity checks, complexity analysis, and test cases. Extend the result to a nested-list format.

```
def topological_order(vertices, edges):
```

5. Number of Islands

MEDIUM | asked 2026-09-30 | grid, dfs, bfs, union-find

Examples: [[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2. Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1] (last addition merges the two land cells).

```
def num_islands(grid):
```

6. Path Minimizing the Maximum Cell (Swim in Rising Water)

MEDIUM | asked 2026-09-29 | graph, heap, algorithm

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold. Grid variant Input is a grid of heights and start/end cells; movement is through four-neighbors. Cost of a path is the maximum cell value along it. Return the minimum possible path cost.

```
def min_max_path(g, start, end):
```

7. Shortest Path Nodes and Escaping a Pursuer

MEDIUM | asked 2026-09-29 | graph, bfs, shortest-path

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node. Return the shortest path (length) from Alice to the target. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

```
def shortest_length(adj, a, t):
```

8. Rotten Oranges / Multi-Source BFS (taxis)

MEDIUM | asked 2026-09-08 | grid, bfs

Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges variant): return number of minutes until all reachable 1s become 2 (4). Taxi variant: for each empty cell, distance to nearest taxi.

```
def rotting_minutes(grid):
```

9. 2D Points Connected Components by Radius

MEDIUM | asked 2026-08-04 | graph, dfs, bfs, union-find, clustering

Given 2D points and radius r, connect two points when their distance is at most r; connectivity is transitive, and the output is the final number of clusters. Input: a list of 2D points and a radius r. Two points are connected if their Euclidean distance is <= r. Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster. Return the number of final clusters.

```
def count_clusters(points, r):
```

10. Minimum Direction Violations in a Directed Graph

MEDIUM | asked 2026-07-19 | graph, bfs, dijkstra

Examples: n = 5 edges = [[0, 1], [2, 1], [2, 3], [4, 3]] start = 0 end = 4 path: 0 -> 1 -> 2 -> 3 -> 4 cost: 0 + 1 + 0 + 1 = 2 output: 2

```
def min_violations(n, edges, start, end):
```

11. Shortest Directed Cycle Through a Node

MEDIUM | asked n/a | graph, bfs

Examples: Edges [(0,1),(1,2),(2,0)], v = 0 -> 3. Edges [(0,1),(1,2)], v = 0 -> -1.

```
def shortest_cycle_through(n, edges, v):
```